

CapMar — Documentação do Backend e do Banco de Dados

Plataforma web para conectar pequenos empreendedores e criadores de projetos de **Maricá** a investidores. Este documento cobre: **(1)** a documentação do backend, **(2)** a modelagem de dados, **(3)** o projeto lógico do banco de dados e **(4)** as imagens do banco criado.

Projeto	CapMar (MVP)
Camada	Backend / API REST
Stack	Python 3.10+ · FastAPI · SQLAlchemy 2.0 · Pydantic v2 · PostgreSQL 16
Entrypoint	<code>app.main:app</code> (Uvicorn ASGI)
Documentação interativa	<code>http://localhost:8000/docs</code> (Swagger UI) · <code>http://localhost:8000/redoc</code>

Sumário

1. Visão geral do backend
2. Arquitetura e organização do código
3. Configuração, segurança e autenticação
4. Documentação da API (endpoints)
5. Modelagem de dados (modelo conceitual)
6. Projeto lógico do banco de dados
7. Imagens do banco criado
8. Como executar

1. Visão geral do backend

O backend é uma **API RESTful** que centraliza toda a regra de negócio da plataforma: cadastro e autenticação de usuários, gestão de projetos, equipes, publicações (feed), curtidas, comentários e upload de imagens. Ele é totalmente desacoplado do frontend (Next.js) e se comunica via JSON sobre HTTP, liberando o acesso por **CORS** para a origem do frontend.

Características principais:

- **Stateless / JWT** — a autenticação usa *tokens* JWT Bearer; não há sessão no servidor.
- **ORM declarativo** — as tabelas são descritas como classes SQLAlchemy em `app/models.py`.
- **Validação na borda** — todo corpo de requisição/resposta passa por *schemas* Pydantic v2.
- **Auto-criação de schema** — na inicialização (`lifespan`) as tabelas faltantes são criadas e as categorias-padrão são semeadas (adequado ao MVP; migrar para **Alembic** quando o schema estabilizar).

2. Arquitetura e organização do código

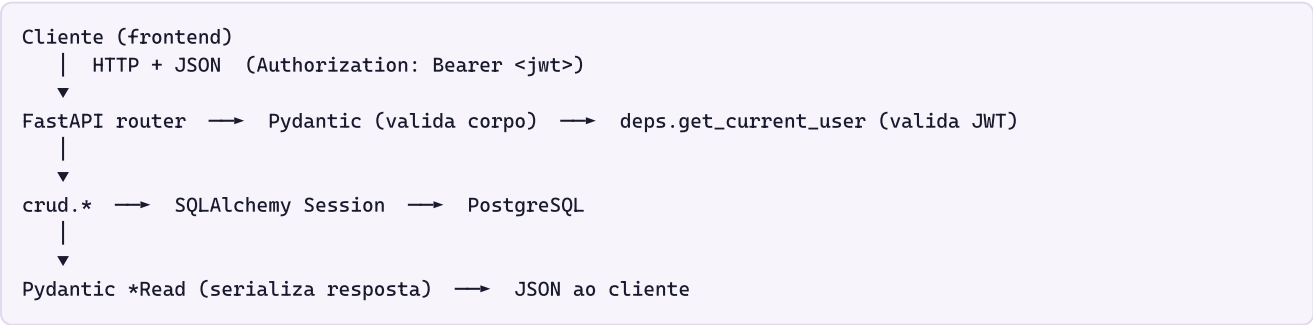
2.1 Estrutura de pastas

```
backend/
├── app/
│   ├── main.py           # cria o app, CORS, lifespan (cria tabelas + seed), monta /uploads e /health
│   ├── config.py         # Settings via variáveis de ambiente / .env (Pydantic Settings)
│   ├── database.py       # engine, SessionLocal, Base declarativa e dependency get_db()
│   ├── models.py         # tabelas do banco (SQLAlchemy 2.0): entidades + tabelas de associação
│   ├── schemas.py        # contratos de entrada/saída da API (Pydantic v2)
│   ├── crud.py           # funções de acesso a dados (camada de persistência)
│   ├── security.py       # hash de senha (bcrypt) e geração/decodificação de JWT
│   ├── deps.py           # dependências de autenticação (get_current_user / _optional)
│   └── routers/          # endpoints agrupados por recurso
│       ├── auth.py       # /auth/register, /auth/login
│       ├── users.py      # /users
│       ├── projects.py   # /projects (+ likes e comentários)
│       ├── teams.py      # /teams (+ membros)
│       ├── posts.py      # /posts (feed)
│       ├── categories.py # /categories
│       └── uploads.py     # /upload (imagens)
├── Database Schema.txt   # fonte original do schema em DBML
├── Dockerfile
└── requirements.txt
```

2.2 Camadas e responsabilidades

Camada	Arquivo(s)	Responsabilidade
Roteamento / HTTP	routers/*.py	Recebe a requisição, valida regras de autorização, traduz erros em <code>HTTPException</code> .
Contrato (DTO)	schemas.py	Define e valida os formatos de entrada (<code>*Create</code> , <code>*Update</code>) e saída (<code>*Read</code>).
Persistência	crud.py	Funções puras de leitura/escrita no banco usando a sessão SQLAlchemy.
Modelo de dados	models.py	Mapeia tabelas, colunas, chaves e relacionamentos (ORM).
Infra	database.py , config.py , security.py , deps.py	Conexão, configuração, autenticação.

2.3 Fluxo de uma requisição



A sessão de banco é aberta por requisição pela dependency `get_db()` (`database.py`) e fechada ao final, garantindo isolamento transacional por requisição.

3. Configuração, segurança e autenticação

3.1 Variáveis de ambiente (`app/config.py`)

Variável	Padrão	Descrição
<code>DATABASE_URL</code>	<code>postgresql+psycopg2://capmar:capmar@localhost:5432/capmar</code>	String de conexão do PostgreSQL.
<code>CORS_ORIGINS</code>	<code>http://localhost:3000</code>	Origens permitidas (separadas por vírgula).
<code>SECRET_KEY</code>	<code>dev-secret-change-me</code>	Chave de assinatura do JWT — trocar em produção .
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	<code>10080</code> (7 dias)	Validade do token de acesso.
<code>UPLOAD_DIR</code>	<code>/app/uploads</code>	Diretório das imagens enviadas (volume no Docker).

3.2 Segurança (`app/security.py`)

- **Senhas:** nunca são armazenadas em texto puro — são processadas com **bcrypt** (`hash_password` / `verify_password`). A senha é truncada em 72 bytes (limite do bcrypt) antes do hash.
- **Tokens:** JWT assinado com **HS256**; o `sub` carrega o `user_id` e há expiração (`exp`).

3.3 Autenticação e autorização (`app/deps.py`)

- `get_current_user` — exige um Bearer válido; responde **401** se ausente/ inválido.
- `get_current_user_optional` — retorna o usuário se houver token, ou `None` (usado em listagens públicas para marcar, por exemplo, `liked_by_me`).
- **Regras de propriedade (ownership):**
 - Projeto: somente o `owner_id` pode editar/excluir (**403** caso contrário).
 - Equipe: somente o `leader_id` pode editar/excluir e gerenciar membros.
 - Comentário: o autor **ou** o dono do projeto podem excluir.

3.4 Upload de imagens (`app/routers/uploads.py`)

POST `/upload` (autenticado, `multipart/form-data`) aceita **JPEG/PNG/WEBP/GIF** até **5 MB**, salva o arquivo com nome aleatório (UUID) em `UPLOAD_DIR` e retorna `{"url": "/uploads/<arquivo>"}` . Os arquivos são servidos estaticamente em `/uploads` via `StaticFiles` .

4. Documentação da API (endpoints)

Base URL local: `http://localhost:8000` . Recursos mutáveis exigem `Authorization: Bearer <token>` . A coluna **Auth** indica:  público ·  exige token ·  /  opcional.

4.1 Autenticação — `/auth`

Método	Rota	Auth	Descrição	Corpo / Resposta
POST	<code>/auth/register</code>		Cria usuário e já retorna o token.	<code>UserCreate</code> → <code>AuthResponse</code>
POST	<code>/auth/login</code>		Autentica por e-mail + senha.	<code>LoginRequest</code> → <code>AuthResponse</code>

`AuthResponse = { access_token, token_type: "bearer", user }` .

4.2 Usuários — /users

Método	Rota	Auth	Descrição
GET	/users		Lista usuários (dados públicos).
GET	/users/{user_id}		Detalha um usuário.
PATCH	/users/{user_id}		Atualiza o próprio perfil (valida e-mail/username únicos).

4.3 Projetos — /projects

Método	Rota	Auth	Descrição
GET	/projects	/	Lista projetos (paginação <code>skip / limit</code>); marca <code>liked_by_me</code> .
POST	/projects		Cria projeto (o autor vira <code>owner</code>).
GET	/projects/{id}	/	Detalha projeto.
PATCH	/projects/{id}	(dono)	Atualização parcial.
DELETE	/projects/{id}	(dono)	Exclui projeto.
POST	/projects/{id}/like		Curte o projeto.
DELETE	/projects/{id}/like		Remove a curtida.
GET	/projects/{id}/comments		Lista comentários.
POST	/projects/{id}/comments		Cria comentário.
DELETE	/projects/{id}/comments/{comment_id}	(autor ou dono)	Exclui comentário.

4.4 Equipes — /teams

Método	Rota	Auth	Descrição
GET	/teams?project_id=		Lista equipes (opcionalmente de um projeto).
POST	/teams		Cria equipe (o autor vira <code>leader</code> e primeiro membro).
GET	/teams/{id}		Detalha equipe (inclui membros).
PATCH	/teams/{id}	(líder)	Atualiza equipe.
DELETE	/teams/{id}	(líder)	Exclui equipe.
POST	/teams/{id}/members	(líder)	Adiciona membro.
DELETE	/teams/{id}/members/{user_id}	(líder)	Remove membro (o líder não pode sair).

4.5 Feed — /posts

Método	Rota	Auth	Descrição
GET	/posts?user_id=		Lista publicações (filtra por autor opcionalmente).
POST	/posts		Cria publicação (pode referenciar um projeto).
DELETE	/posts/{id}	(autor)	Exclui publicação.

4.6 Categorias e utilitários

Método	Rota	Auth	Descrição
GET	/categories		Lista categorias.
POST	/categories		Cria categoria.
POST	/upload		Envia uma imagem (≤ 5 MB) e retorna sua URL.
GET	/health		Health check ({ "status": "ok" }).

5. Modelagem de dados (modelo conceitual)

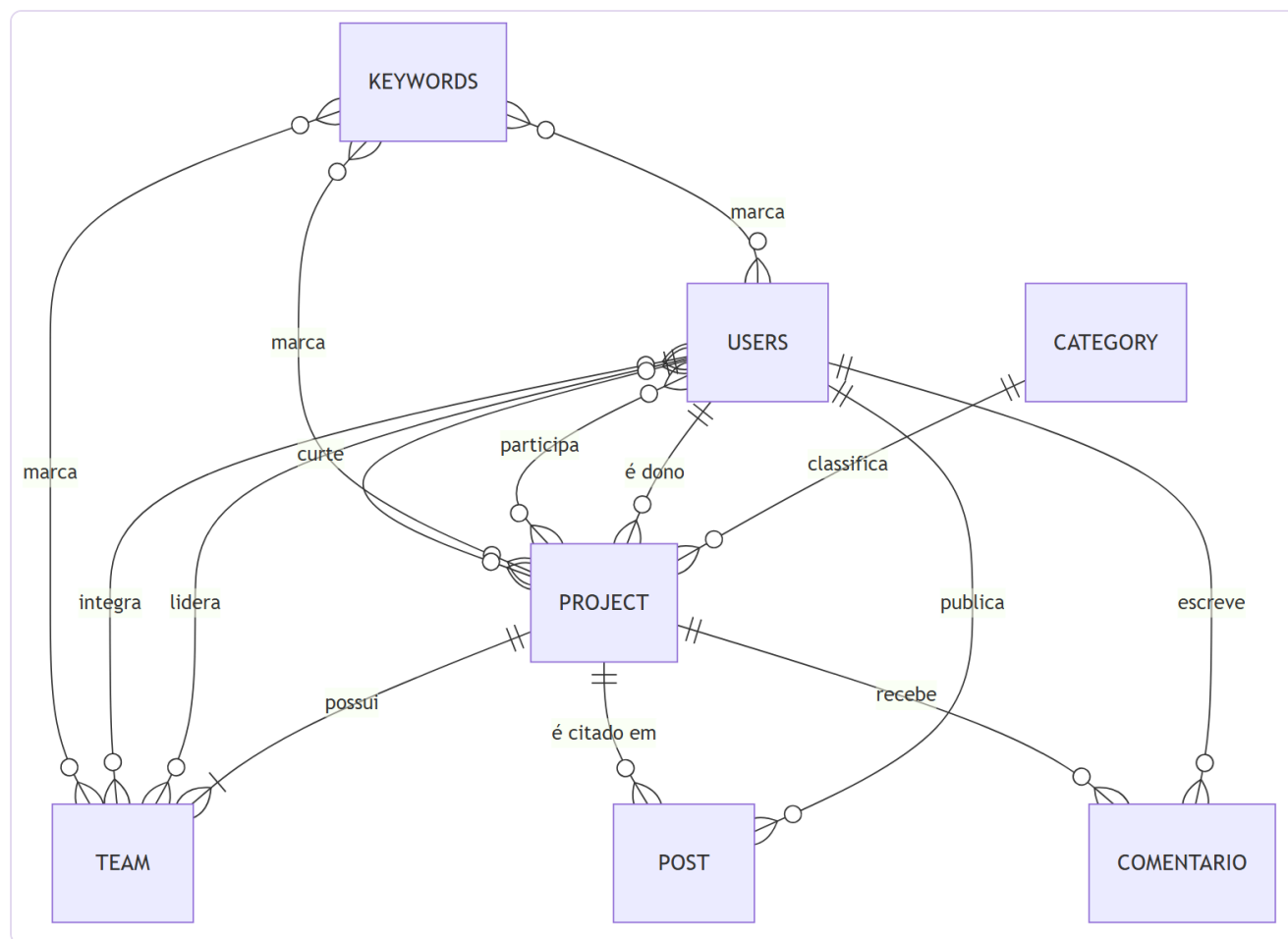
5.1 Entidades de negócio

Entidade	Descrição
USERS	Pessoa cadastrada (empreendedor, criador ou investidor).
PROJECT	Projeto/empreendimento publicado na plataforma.
TEAM	Equipe de trabalho vinculada a um projeto.
CATEGORY	Categoria/segmento que classifica um projeto (Gastronomia, Tecnologia...).
KEYWORDS	Palavras-chave/tags de descoberta, com popularity .
POST	Publicação no feed, opcionalmente associada a um projeto.
COMENTÁRIO / CURTIDA	Interações sociais sobre projetos.

5.2 Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Observação
CATEGORY → PROJECT	1 : N	Um projeto pertence a no máximo uma categoria.
USERS → PROJECT (dono)	1 : N	owner_id — autor/proprietário do projeto.
PROJECT → TEAM	1 : N	Um projeto pode ter várias equipes.
USERS → TEAM (líder)	1 : N	leader_id — líder da equipe.
USERS ↔ PROJECT (membros)	N : N	Tabela project_users .
USERS ↔ TEAM (membros)	N : N	Tabela team_users .
USERS ↔ PROJECT (curtidas)	N : N	Tabela project_likes .
KEYWORDS ↔ PROJECT / TEAM / USERS	N : N	projects_keywords , team_keywords , user_keywords .
USERS → POST	1 : N	Autoria da publicação.
PROJECT → POST	1 : N	Publicação pode citar um projeto.
USERS / PROJECT → COMENTÁRIO	1 : N	Comentários de projeto.

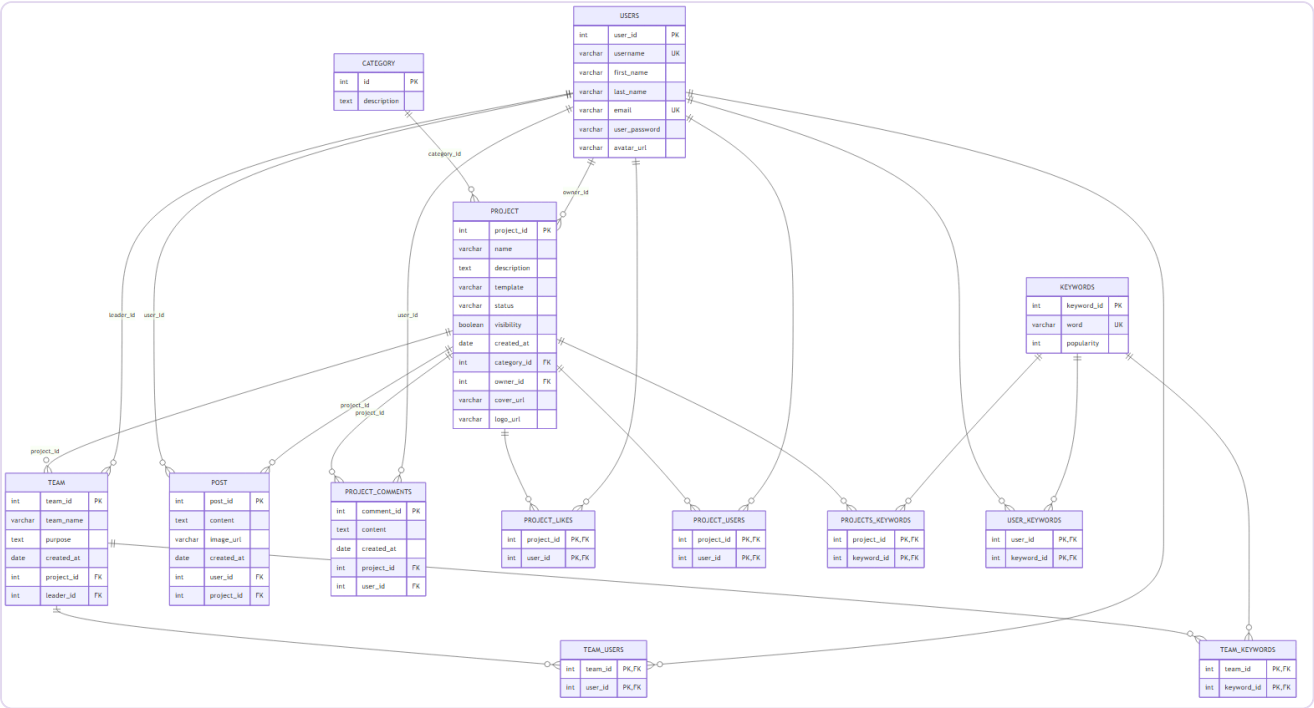
5.3 Diagrama Entidade-Relacionamento (conceitual)



O diagrama acima é renderizado automaticamente como imagem no GitHub e no preview de Markdown do VS Code. Para exportar como arquivo `.png` / `.svg`, veja a [seção 7](#).

6. Projeto lógico do banco de dados

6.1 Diagrama lógico/físico (com atributos)



6.2 Dicionário de dados (tabelas e colunas)

Legenda: **PK** = chave primária · **FK** = chave estrangeira · **UK** = único · **NN** = *not null*.

users — usuários

Coluna	Tipo	Restrições	Descrição
user_id	SERIAL	PK	Identificador.
username	VARCHAR(20)	UK, NN	Nome de usuário.
first_name	VARCHAR(20)	—	Nome.
last_name	VARCHAR(100)	—	Sobrenome.
email	VARCHAR(50)	UK, NN	E-mail.
user_password	VARCHAR(255)	NN	Hash bcrypt da senha.
avatar_url	VARCHAR(255)	—	URL do avatar.

category — categorias

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PK	Identificador.
description	TEXT	NN	Nome/descrição da categoria.

project — projetos

Coluna	Tipo	Restrições	Descrição
project_id	SERIAL	PK	Identificador.
name	VARCHAR(100)	NN	Nome do projeto.
description	TEXT	—	Descrição.
template	VARCHAR(100)	—	Modelo/template visual.
status	VARCHAR(15)	NN, default 'rascunho'	Situação.
visibility	BOOLEAN	NN, default true	Público/privado.
created_at	DATE	NN, default CURRENT_DATE	Data de criação.
category_id	INTEGER	FK → category(id) ON DELETE SET NULL	Categoria.
owner_id	INTEGER	FK → users(user_id) ON DELETE SET NULL	Dono.
cover_url	VARCHAR(255)	—	Imagem de capa.
logo_url	VARCHAR(255)	—	Logotipo.

team — equipes

Coluna	Tipo	Restrições	Descrição
team_id	SERIAL	PK	Identificador.
team_name	VARCHAR(255)	NN	Nome da equipe.
purpose	TEXT	—	Propósito/objetivo.
created_at	DATE	NN, default CURRENT_DATE	Data de criação.
project_id	INTEGER	FK → project(project_id) ON DELETE CASCADE , NN	Projeto.
leader_id	INTEGER	FK → users(user_id) ON DELETE SET NULL	Líder.

keywords — palavras-chave

Coluna	Tipo	Restrições	Descrição
keyword_id	SERIAL	PK	Identificador.
word	VARCHAR(20)	UK, NN	Termo.
popularity	INTEGER	NN, default 0	Popularidade/uso.

post — publicações do feed

Coluna	Tipo	Restrições	Descrição
post_id	SERIAL	PK	Identificador.
content	TEXT	NN	Texto da publicação.
image_url	VARCHAR(255)	—	Imagem anexa.
created_at	DATE	NN, default CURRENT_DATE	Data.
user_id	INTEGER	FK → users(user_id) ON DELETE CASCADE , NN	Autor.
project_id	INTEGER	FK → project(project_id) ON DELETE SET NULL	Projeto citado.

project_comments — comentários de projeto

Coluna	Tipo	Restrições	Descrição
comment_id	SERIAL	PK	Identificador.
content	TEXT	NN	Texto.
created_at	DATE	NN, default CURRENT_DATE	Data.
project_id	INTEGER	FK → project(project_id) ON DELETE CASCADE , NN	Projeto.
user_id	INTEGER	FK → users(user_id) ON DELETE CASCADE , NN	Autor.

Tabelas de associação (N:N) e curtidas

Todas têm **chave primária composta** e `ON DELETE CASCADE` nas duas FKs.

Tabela	Colunas (PK composta)	Liga
project_likes	(project_id, user_id)	curtidas de projeto
project_users	(project_id, user_id)	membros do projeto
team_users	(team_id, user_id)	membros da equipe
projects_keywords	(project_id, keyword_id)	tags do projeto
team_keywords	(team_id, keyword_id)	tags da equipe
user_keywords	(user_id, keyword_id)	tags do usuário

6.3 Observações sobre normalização

- O modelo está na **3ª Forma Normal (3FN)**: cada tabela tem chave primária própria, não há grupos repetitivos e os atributos dependem apenas da chave.
- Os relacionamentos **N:N** são resolvidos por **tabelas associativas** com chave primária composta, evitando redundância.
- As ações de integridade referencial são deliberadas: `CASCADE` quando o filho não faz sentido sem o pai (ex.: equipes de um projeto, vínculos N:N) e `SET NULL` quando o registro deve sobreviver à remoção da referência (ex.: `owner_id` / `leader_id` / `category_id`).

6.4 DDL — script de criação (PostgreSQL)

Equivalente SQL ao que o SQLAlchemy gera em `Base.metadata.create_all()`. Veja também o arquivo executável `docs/schema.sql`.

```

CREATE TABLE category (
    id SERIAL PRIMARY KEY,
    description TEXT NOT NULL
);

CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(20) NOT NULL UNIQUE,
    first_name VARCHAR(20),
    last_name VARCHAR(100),
    email VARCHAR(50) NOT NULL UNIQUE,
    user_password VARCHAR(255) NOT NULL,
    avatar_url VARCHAR(255)
);

CREATE TABLE keywords (
    keyword_id SERIAL PRIMARY KEY,
    word VARCHAR(20) NOT NULL UNIQUE,
    popularity INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE project (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    template VARCHAR(100),
    status VARCHAR(15) NOT NULL DEFAULT 'rascunho',
    visibility BOOLEAN NOT NULL DEFAULT TRUE,
    created_at DATE NOT NULL DEFAULT CURRENT_DATE,
    category_id INTEGER REFERENCES category(id) ON DELETE SET NULL,
    owner_id INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
    cover_url VARCHAR(255),
    logo_url VARCHAR(255)
);

CREATE TABLE team (
    team_id SERIAL PRIMARY KEY,
    team_name VARCHAR(255) NOT NULL,
    purpose TEXT,
    created_at DATE NOT NULL DEFAULT CURRENT_DATE,
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,
    leader_id INTEGER REFERENCES users(user_id) ON DELETE SET NULL
);

CREATE TABLE post (
    post_id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    image_url VARCHAR(255),
    created_at DATE NOT NULL DEFAULT CURRENT_DATE,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    project_id INTEGER REFERENCES project(project_id) ON DELETE SET NULL
);

CREATE TABLE project_comments (
    comment_id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    created_at DATE NOT NULL DEFAULT CURRENT_DATE,
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE
);

CREATE TABLE project_likes (
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    PRIMARY KEY (project_id, user_id)
);

CREATE TABLE project_users (
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,

```

```

    user_id    INTEGER NOT NULL REFERENCES users(user_id)      ON DELETE CASCADE,
    PRIMARY KEY (project_id, user_id)
);

CREATE TABLE team_users (
    team_id INTEGER NOT NULL REFERENCES team(team_id) ON DELETE CASCADE,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    PRIMARY KEY (team_id, user_id)
);

CREATE TABLE projects_keywords (
    project_id INTEGER NOT NULL REFERENCES project(project_id) ON DELETE CASCADE,
    keyword_id INTEGER NOT NULL REFERENCES keywords(keyword_id) ON DELETE CASCADE,
    PRIMARY KEY (project_id, keyword_id)
);

CREATE TABLE team_keywords (
    team_id    INTEGER NOT NULL REFERENCES team(team_id)      ON DELETE CASCADE,
    keyword_id INTEGER NOT NULL REFERENCES keywords(keyword_id) ON DELETE CASCADE,
    PRIMARY KEY (team_id, keyword_id)
);

CREATE TABLE user_keywords (
    user_id    INTEGER NOT NULL REFERENCES users(user_id)      ON DELETE CASCADE,
    keyword_id INTEGER NOT NULL REFERENCES keywords(keyword_id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, keyword_id)
);

```

Nota (MVP): no código, as tabelas são criadas por `Base.metadata.create_all()` no *startup*. Como `create_all()` **não** altera tabelas existentes, colunas adicionadas depois (ex.: `owner_id`, `avatar_url`, `cover_url`, `logo_url`, `image_url`, `leader_id`) são garantidas por `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` em `ensure_schema()` (`app/main.py`). A migração definitiva está prevista para **Alembic**.

7. Imagens do banco criado

Os diagramas Entidade-Relacionamento estão disponíveis em três formatos, para diferentes usos:

Formato	Arquivo	Como visualizar / exportar
Diagrama ER (modelagem)	<code>docs/diagramas/DRE_capmar.jpg</code>	Diagrama ER formal do projeto (imagem do banco modelado).
PNG (schema implementado)	<code>docs/diagramas/modelo-logico.png</code> , <code>modelo-conceitual.png</code>	Imagens prontas para colar em relatórios.
Markdown (Mermaid)	este documento (seções 5.3 e 6.1)	Renderiza como imagem no GitHub e no preview do VS Code.
HTML interativo	<code>docs/diagrama-er.html</code>	Abra no navegador e clique em Baixar PNG/SVG .
Fonte Mermaid	<code>docs/diagramas/modelo-conceitual.mmd</code> , <code>modelo-logico.mmd</code>	Editáveis; renderizáveis com <code>mermaid-cli</code> .

7.1 Como obter os arquivos de imagem (PNG/SVG)

Opção A — pelo navegador (sem instalar nada): 1. Abra `docs/diagrama-er.html` no navegador. 2. Clique em **Baixar PNG** (ou **Baixar SVG**) no diagrama desejado.

Opção B — via linha de comando (mermaid-cli):

```
npx -y @mermaid-js/mermaid-cli -i docs/diagramas/modelo-logico.mmd \
-o docs/diagramas/modelo-logico.png -b white -s 2
```

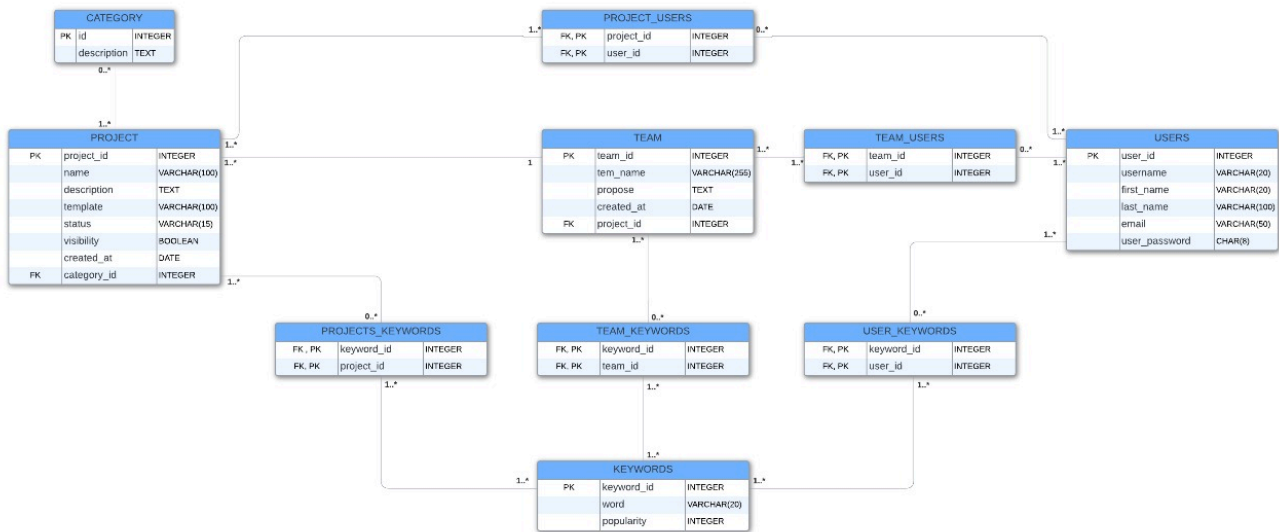
Opção C — diagrama editável (dbdiagram.io): Copie o conteúdo de `backend/Database Schema.txt` (formato DBML) e cole em <https://dbdiagram.io> para gerar/editar o diagrama visual.

7.2 Imagens do banco

(a) Diagrama ER do projeto — modelagem original

Diagrama Entidade-Relacionamento formal elaborado na modelagem do banco (fonte: `backend/Database Schema.txt`). Mostra as 5 entidades principais (CATEGORY, PROJECT, TEAM, USERS, KEYWORDS) e as 5 tabelas associativas. *Observação:* dois nomes foram corrigidos na implementação (`tem_name` → `team_name`, `propose` → `purpose`) e o tipo de `user_password` foi ampliado para `VARCHAR(255)` (hash bcrypt).

Entity Relationship Diagram - Capmar



(b) Modelo lógico implementado — schema atual

Schema efetivamente criado pelo backend (inclui as evoluções do MVP: `owner_id`, `leader_id`, `cover_url`, `logo_url`, `avatar_url` e as tabelas `post`, `project_likes`, `project_comments`).

8. Como executar

8.1 Via Docker (recomendado — sobe API + Postgres juntos)

A partir da **raiz do repositório**:

```
cp .env.example .env          # ajuste as variáveis se necessário
docker compose up --build
```

- API: <http://localhost:8000> · Swagger: <http://localhost:8000/docs>
- PostgreSQL: `localhost:5432`

8.2 Localmente (sem Docker)

Requer um PostgreSQL 16 acessível e a `DATABASE_URL` configurada em `backend/.env`.

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

8.3 Verificação rápida

```
curl http://localhost:8000/health      # {"status":"ok"}
curl http://localhost:8000/categories  # categorias semeadas no startup
```

Documento gerado para o projeto CapMar (Univassouras / PEI). Fonte do schema: `backend/models.py` e `backend/Database Schema.txt`.